

University of Yahia Fares Medea

Faculty of Sciences & Technology

Department of Computer Science

Academic Year 2025 / 2026

Go · Python · NeMo FastConformer · Docker · Caddy

Tahkik Inference Server

Technical Report

Server Architecture, API & Deployment

Benhadjer Mohamed Bouziani Alaa Eddine

Group 2

Group 2

L3 Computer Science

L3 Computer Science

Contents

1	Introduction	4
2	System Architecture	5
2.1	System Overview	5
2.2	Design Justification	5
3	Component Details	7
3.1	Caddy Reverse Proxy	7
3.1.1	Overview	7
3.1.2	Routing Rules	7
3.2	Go API Server	7
3.2.1	Overview	7
3.2.2	Technology	8
3.2.3	Key Implementation Details	8
3.2.4	Inference Client	9
3.2.5	System-Health Aggregator	10
3.3	Python Inference Server	10
3.3.1	Overview	10
3.3.2	Technology	10
3.3.3	Model Loading	11
3.3.4	Inference Wrapper and Confidence Scoring	11
3.3.5	Inference Concurrency Lock	12
3.3.6	WebSocket Streaming Protocol	12
4	Data Pipeline	14
4.1	Audio Processing Flow	14
4.2	PCM Conversion (WebSocket Path)	15
4.3	Confidence Scoring	15
5	Madd (Tajweed) Analysis	16
5.1	Goal	16
5.2	Rules Covered	16
5.3	Pipeline	17
5.4	Waqf Detection	17

5.5	Output Schema	18
6	Setup and Deployment	19
6.1	Production: One-Shot Droplet Provisioning	19
6.2	Continuous Deployment	19
6.3	Local Development	20
6.4	Environment Variables	21
6.5	Hardware Recommendations	21
7	API Reference	22
7.1	Go Server Endpoints (Client-Facing)	22
7.1.1	POST /api/v1/evaluate	22
7.1.2	GET /api/v1/system-health (Bearer-token gated)	23
7.1.3	GET /api/v1/system-health/ws (WebSocket)	24
7.1.4	GET /health	24
7.2	Response Fields	24
7.3	Flutter Integration Example	24
8	Troubleshooting	26
8.0.1	Inference Server Unreachable / 502 from Caddy	26
8.0.2	Model Download Timeout	26
8.0.3	system-health Returns 503 / 401	26
8.0.4	Partial Latency Above the Window	27
9	Performance Metrics	28
9.1	Inference Latency by Hardware	28
9.2	Processing Time Breakdown	28
10	Scalability and Load Handling	30
10.1	Current Concurrency Model	30
10.2	Observed Behaviour Under Load	30
10.3	Scalability Improvement Strategies	31
11	Security Considerations	32
11.1	Transport Layer	32
11.2	Input Validation	32
11.3	Token Management	32
11.4	Docker Socket Exposure	33
11.5	Abuse Prevention	34
12	Request Lifecycle	35

12.1 End-to-End Request Flow	35
12.2 Concurrency Patterns	36
13 Failure Handling	37
13.1 Error Categories and Responses	37
13.2 Error Response Format	37
13.3 Resilience Mechanisms	38
14 Logging and Monitoring	39
14.1 Logging Strategy	39
14.2 Metrics Exposure	39
14.3 Observability Roadmap	40
14.4 Dashboard Integration	40
15 Conclusion	41
15.1 Summary	41
15.2 Key Achievements	41
15.3 Future Enhancements	42
15.4 Final Remarks	42
References	43

1. Introduction

This technical report documents the server-side infrastructure of the Tahkik project — an AI-powered Quranic recitation analysis system specialising in the Warsh ‘an Nafi’ riwaya (Tariq al-Azraq). The server bridges the Flutter mobile application with the speech-recognition model, accepts audio uploads (or live PCM streams), runs an optional Tajweed (madd) analysis against a reference text, and returns structured transcription and error results.

The production stack is composed of three containerised components running side-by-side on a single Linux host: a **Caddy** reverse proxy (auto-TLS via Let’s Encrypt), a **Go** HTTP API gateway, and a **Python FastAPI** inference server backed by the NVIDIA NeMo **FastConformer Hybrid CTC/Transducer** model (`nvidia/stt_ar_fastconformer_hybrid_large_pcd_v1.0`). A WebSocket streaming endpoint (`/ws/stream`) delivers real-time partial transcription, and a separate `/api/v1/system-health` endpoint (plus its WebSocket variant) exposes host metrics, container status, and inference latency for the operations dashboard.

This report covers the architecture, component details, data pipeline, madd analyser, API contracts, deployment procedure, security model, and operational concerns of the server.

2. System Architecture

2.1 System Overview

The Tahkik server runs as a three-container Docker Compose stack. Caddy terminates TLS and routes traffic; the Go API server handles HTTP REST traffic and system-health aggregation; the Python container runs the NeMo FastConformer model and also serves the live WebSocket stream. The Flutter client never speaks to the Python container directly — it talks to the public hostname which Caddy proxies appropriately.

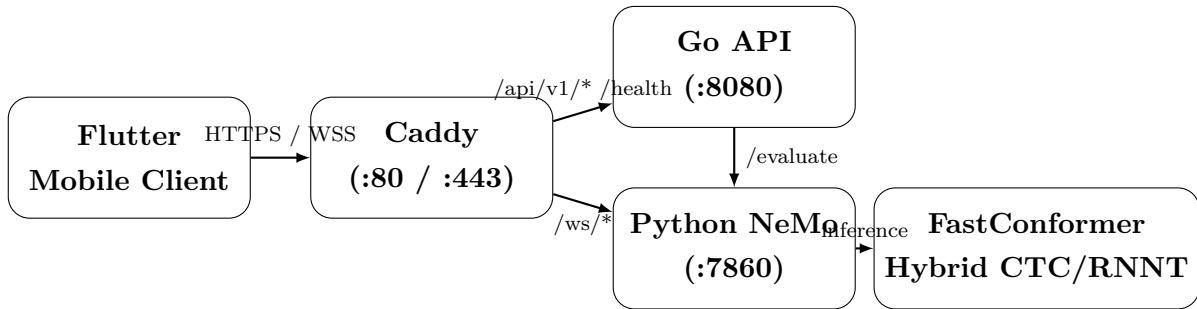


Figure 2.1: High-level system architecture

2.2 Design Justification

The architectural decisions were driven by the following constraints:

- **Reverse proxy at the edge:** Caddy handles automatic TLS issuance from Let's Encrypt, CORS preflight for the admin dashboard, and routing rules (`/api/v1/*` and `/health` to the Go server; `/ws/*` straight to the Python container). This isolates concerns and lets the Go layer focus on JSON, not TLS.
- **Go for the API gateway:** Go's standard library `net/http` delivers a production-ready server with minimal dependencies, fast static-linked builds, and a tiny distro-less image (`gcr.io/distroless/static-debian12:nonroot`, ~10 MB). The single external dependency (`github.com/coder/websocket`) is used only by the system-health stream.
- **NeMo FastConformer for inference:** The NVIDIA Arabic ASR checkpoint achieves 6.55 % WER on the EveryAyah Quran benchmark and ships a hybrid CTC

+ RNNT head. The RNNT head is used for transcription; the CTC head is reused for forced alignment against a reference text (see Chapter 5).

- **Single-host deployment:** The full stack runs on one DigitalOcean droplet provisioned by `scripts/provision-droplet.sh`. CPU-only droplets work out of the box; a GPU droplet is enabled by a one-line overlay (`docker-compose.gpu.yml`) which sets `NEMO_DEVICE=cuda` and reserves the NVIDIA device.
- **WebSocket streaming:** The `/ws/stream` endpoint enables real-time partial transcription with an 8-second sliding window, giving the reciter immediate feedback without waiting for the recording to finish.

3. Component Details

3.1 Caddy Reverse Proxy

3.1.1 Overview

Caddy is the only container exposed to the public internet. It listens on ports 80 and 443, automatically issues a TLS certificate from Let's Encrypt on the first HTTPS request, and routes incoming traffic to the appropriate backend container.

3.1.2 Routing Rules

The full Caddyfile matches three classes of request:

Listing 3.1: Caddy routing (excerpt from Caddyfile)

```
1 @api_preflight { method OPTIONS path /api/v1/* /health }
2 @api           { path /api/v1/* /health }
3 @ws            { path /ws/* }
4
5 handle @api { reverse_proxy go-server:8080 }
6 handle @ws { reverse_proxy python:7860 }
```

The OPTIONS matcher injects the CORS headers (Access-Control-Allow-Origin: \$DASHBOARD_ORIGIN) so the dashboard can call the API from a browser. WebSocket frames bypass the Go layer entirely.

3.2 Go API Server

3.2.1 Overview

The Go API server is the entry point for all REST traffic. It listens on port 8080 (configurable via PORT) and exposes four endpoints: POST /api/v1/evaluate for audio evaluation, GET /api/v1/system-health for aggregated metrics, GET /api/v1/system-health/ws for the real-time stream, and GET /health for liveness checks. A fifth healthcheck sub-command exists so the distroless Docker image can run its own HEALTHCHECK without needing curl.

3.2.2 Technology

Technology Stack

- **Language:** Go 1.26.2
- **Framework:** Standard library (net/http)
- **WebSocket:** github.com/coder/websocket v1.8.14
- **Image:** gcr.io/distroless/static-debian12:nonroot (statically linked, ~10 MB)

3.2.3 Key Implementation Details

The server uses Go's built-in `http.NewServeMux` for routing. The `Evaluate` handler validates the uploaded audio file (max 50 MB, supported formats: `.wav`, `.mp3`, `.m4a`, `.flac`, `.ogg`), reads the binary data, optionally captures a `reference_text` form field, and delegates to the ML inference client.

Listing 3.2: API entry point (`cmd/api/main.go`)

```

1 func main() {
2     if len(os.Args) > 1 && os.Args[1] == "healthcheck" {
3         healthcheckMain()
4         return
5     }
6     port := os.Getenv("PORT")
7     if port == "" { port = "8080" }
8
9     log.Printf("inference_backend: %s", ml.InferenceServerURL())
10    log.Printf("version: git_sha=%s_deployed_at=%s",
11        handlers.GitSHA, handlers.DeployedAt)
12
13    mux := http.NewServeMux()
14    mux.HandleFunc("/api/v1/evaluate", handlers.Evaluate)
15    mux.HandleFunc("/api/v1/system-health", requireBearer(
16        handlers.SystemHealth))
17    mux.HandleFunc("/api/v1/system-health/ws", handlers.
18        SystemHealthWS)
19    mux.HandleFunc("/health", func(w http.ResponseWriter, r *http
20        .Request) {
21        w.Header().Set("Content-Type", "application/json")
22        _, _ = w.Write([]byte(`{"status":"ok"}`))
23    })
24    log.Printf("Tahkik_server_listening_on: %s", port)

```

```

22     http.ListenAndServe(":"+port, mux)
23 }

```

3.2.4 Inference Client

The `internal/ml/inference.go` module forwards audio to the Python backend as `multipart/form-data`, optionally including a `reference_text` field. When that field is present the inference server runs madd analysis and the returned `Result` includes `errors[]` and `haraka_ms` alongside the transcription.

Listing 3.3: Inference result structure

```

1  type Result struct {
2      Transcription      string          'json:"transcription"'
3      ConfidenceScore    float64         'json:"confidence_score"'
4      ProcessingTimeMs   int             'json:"processing_time_ms"'
5      HarakaMs           int             'json:"haraka_ms,omitempty"'
6      Errors             []TajweedError 'json:"errors,omitempty"'
7  }
8
9  type TajweedError struct {
10     Word          string          'json:"word"'
11     WordIndex     int             'json:"word_index"'
12     Rule          string          'json:"rule"'
13     SubRule       string          'json:"sub_rule"'
14     Severity      string          'json:"severity"'
15     Expected      string          'json:"expected"'
16     Detected      string          'json:"detected"'
17     Explanation   string          'json:"explanation"'
18     Timestamp     *AudioTimestamp 'json:"timestamp,omitempty"'
19 }

```

Key characteristics:

- HTTP client timeout: 5 minutes (accommodates large audio files and NeMo's RNNT decoder latency on CPU)
- Sends Authorization: Bearer {HF_TOKEN} header when the env var is set
- Parses error responses from both `detail` (FastAPI) and `error` (legacy) JSON fields

3.2.5 System-Health Aggregator

GET `/api/v1/system-health` returns a single JSON blob composed from three sources, cached for 5s to avoid thrashing `/proc` and the Docker socket:

- **Host metrics** — read from `/host/proc/*` (uptime, loadavg, meminfo, cpu stat) and `statfs(/host)`. The compose file bind-mounts the host root read-only at `/host` so the Go container observes *real* host values instead of cgroup-virtualised ones.
- **Container status** — enumerated via the Docker Engine API over a read-only bind-mount of `/var/run/docker.sock`. Containers are filtered by the `com.docker.compose.project` label.
- **Inference + model snapshot** — fetched from GET `python:7860/health`, which returns request counters, p50/p95 latency, and the loaded model identifier.

Bearer-token auth is enforced by the `requireBearer` middleware against the `HEALTH_TOKEN` environment variable. If `HEALTH_TOKEN` is unset the endpoint fails closed with 503 `Service Unavailable` rather than going silently public. The WebSocket variant (`/api/v1/system-health/ws`) pushes a fresh snapshot every 2s; auth is via a `?token=` query parameter since browsers cannot set custom headers on a WebSocket handshake.

3.3 Python Inference Server

3.3.1 Overview

The Python container loads the NeMo FastConformer model once at startup and serves three endpoints on port 7860:

- GET `/health` — liveness + model + metrics snapshot
- POST `/evaluate` — batch transcription (with optional madd analysis)
- WS `/ws/stream` — real-time streaming transcription

3.3.2 Technology

Technology Stack

- **Language:** Python 3.10
- **Framework:** FastAPI + Uvicorn
- **ML runtime:** NVIDIA NeMo (`nemo_toolkit[asr]` 2.0.0) on PyTorch ≥ 2.2
- **Model:** `nvidia/stt_ar_fastconformer_hybrid_large_pcd_v1.0`

(~115 M params, Hybrid CTC/RNNT)

- **Audio I/O:** librosa, soundfile, python-multipart
- **Base image:** nvidia/cuda:12.4.1-cudnn-runtime-ubuntu22.04

3.3.3 Model Loading

The server prints `[inference] model ready (warmup <ms>)` when the model is fully loaded, which the system-health endpoint then exposes via the `model.loaded` boolean. Because NeMo 2.x's `transcribe()` cleanup path calls `encoder.unfreeze(partial=True)` — which raises if the module was never frozen — the model is explicitly frozen after `eval()`.

Listing 3.4: Model loading at module import

```

1 MODEL_ID      = os.environ.get(
2     "MODEL_ID",
3     "nvidia/stt_ar_fastconformer_hybrid_large_pcd_v1.0",
4 )
5 NEMO_DEVICE = os.environ.get("NEMO_DEVICE", "cpu").lower()
6
7 import nemo.collections.asr as nemo_asr
8
9 model = nemo_asr.models.EncDecHybridRNNTCTCBPEModel.
    from_pretrained(
10     model_name=MODEL_ID,
11 )
12 model = model.to(NEMO_DEVICE)
13 model.eval()
14 model.freeze()    # required for NeMo 2.x cleanup

```

3.3.4 Inference Wrapper and Confidence Scoring

`_transcribe_array(audio, *, with_timestamps=False)` centralises the call into NeMo and handles all three possible return shapes of `transcribe()` (tuple of lists, list of Hypothesis, or list of strings). Confidence is derived from `Hypothesis.score` (an RNNT log-prob sum) via a soft exponential clamp so it lands in `[0, 1]`:

Listing 3.5: RNNT score → confidence mapping

```

1 def _score_to_confidence(score) -> float:
2     if score is None:
3         return 1.0

```

```

4  return float(np.clip(np.exp(min(0.0, float(score) / 50.0)),
    0.0, 1.0))

```

3.3.5 Inference Concurrency Lock

A module-level `asyncio.Semaphore(1)` caps total concurrent inferences across all connections to avoid CPU over-subscription on small droplets. Each WebSocket connection additionally holds a per-connection `asyncio.Lock` so a single client's partial-window tasks serialise cleanly.

3.3.6 WebSocket Streaming Protocol

The `/ws/stream` endpoint preserves the exact wire-protocol used during the Whisper era so the Flutter application only needs to swap its `HF_SPACE_URL` environment variable to migrate.

Table 3.1: Client → Server messages

Frame	Type	Description
Binary	PCM audio	Raw 16-bit signed LE, 16 kHz, mono
Text	<code>{"type":"stop"}</code>	End of recording — emit final result
Text	<code>{"type":"ping"}</code>	Heartbeat keepalive
Text	<code>{"type":"reset"}</code>	Reset session state
Text	<code>{"type":"context","text":"..."}</code>	Accepted for compat, ignored : NeMo has no <code>initial_prompt</code> biasing equivalent

Table 3.2: Server → Client messages

Type	Description
<code>partial</code>	Intermediate transcription (<code>text</code> field)
<code>final</code>	Final result (<code>text</code> , <code>confidence</code> , <code>processing_time_ms</code>)
<code>error</code>	Error message (<code>message</code> field)
<code>pong</code>	Heartbeat response

Streaming Implementation Highlights

- **Sliding window:** Each partial transcribes the last `PARTIAL_WINDOW_S` = 8 seconds of audio as one contiguous window, avoiding word-cutting at chunk boundaries.
- **Final window:** On `stop`, the last `FINAL_WINDOW_S` = 60 seconds (NeMo handles much longer audio than Whisper's 30s) are re-transcribed for the final, higher-confidence result.
- **Per-connection + global locks:** One inference at a time inside a connection; one at a time across all connections.
- **Generation counter:** Each `reset` increments a counter; in-flight tasks compare their generation before emitting and discard stale output silently.
- **Silence drop:** After 10s of pure silence the audio buffer is dropped to prevent unbounded memory growth.
- **Exception isolation:** Errors inside partial-window tasks are logged with full traceback and do not tear down the WebSocket connection.

4. Data Pipeline

4.1 Audio Processing Flow

When an audio file arrives at the server, it passes through the following stages:

Phase 1 — Audio Reception

The Go server receives the multipart upload, validates the file extension and size (max 50 MB), reads the binary payload into memory, optionally extracts the `reference_text` form field, and forwards both to the Python backend.

Phase 2 — Audio Preprocessing

The Python backend writes the bytes to a temp file under `/tmp`, loads it via `librosa.load(sr=16000, mono=True)`, and clamps the duration to the last 60 seconds. Unlike Whisper, NeMo FastConformer handles long audio directly, so the legacy 30-second chunking step is no longer required.

Phase 3 — Inference

The audio array is handed to `model.transcribe(...)`. When a `reference_text` is present, the call is invoked with `return_hypotheses=True`, `timestamps=True` so the wrapper can extract per-word and per-character timestamps from the returned `Hypothesis` object.

Phase 4 — Optional Madd Analysis

When `reference_text` is provided, a separate CTC forced-alignment pass (Chapter 5) produces per-letter timings, which are then fed into the madd analyser. The analyser returns `errors[]` and an auto-calibrated `haraka_ms`.

Phase 5 — Result Assembly

The transcription, confidence, elapsed time, and (optionally) madd errors are serialised as JSON and returned to the Go server, which wraps it in the standard `{"status": "success", "data": {...}}` envelope.

4.2 PCM Conversion (WebSocket Path)

The streaming path skips `librosa` entirely. Raw PCM bytes are converted to `float32` in-place:

Listing 4.1:]PCM 16-bit signed LE $\rightarrow$ float32 [-1, 1]

```

1 def _pcm_bytes_to_float32(pcm_bytes: bytes) -> np.ndarray:
2     int16_array = np.frombuffer(pcm_bytes, dtype=np.int16)
3     return int16_array.astype(np.float32) / 32768.0

```

A simple RMS-based silence detector (`SILENCE_THRESHOLD = 0.02`) skips inference on quiet windows and triggers the buffer-drop path after sustained silence.

4.3 Confidence Scoring

Confidence Calculation

The NeMo RNNT decoder returns a `Hypothesis.score` that is the sum of log-probabilities over the emitted token sequence (a negative scalar). The server converts it to a $[0, 1]$ confidence via:

$$\text{conf} = \text{clip}\left(\exp\left(\min\left(0, \frac{\text{score}}{50}\right)\right), 0, 1\right)$$

The divisor and clamp were tuned so typical correctly-recited ayahs score above 0.9 and short, low-confidence partials still fall comfortably below 1.0. Values are monotonic but not directly comparable to the Whisper-era `avg_logprob`-based confidence.

5. Madd (Tajweed) Analysis

5.1 Goal

Given a reference text the reciter is expected to say (Warsh orthography) and the audio they actually produced, the madd analyser detects whether each expected vowel-elongation (*madd*) was held for the correct number of counts (*harakat*), grading mistakes as *minor* or *major*. It targets the Warsh ‘an Nafi’ riwaya via Tariq al-Azraq.

5.2 Rules Covered

Table 5.1: Madd rules with expected counts and acceptance windows

Rule (key)	Description	Expected	Window
tabii	Natural madd (no hamza, no sukun)	2	1.5–2.7
badal	Hamza precedes madd letter (qasr)	2	1.5–2.7
muttasil	Madd + hamza, same word	6	5.0–7.5
munfasil	Madd at end of word, hamza next	6	5.0–7.5
lazim_muthaqqal	Madd + shaddah	6	5.0–7.5
lazim_mukhaffaf	Madd + permanent sukun	6	5.0–7.5
aarid	Madd near end of word at waqf (qasr)	2	1.5–2.7
lin	Lin letter at waqf (qasr)	2	1.5–2.7

Warsh/Azraq permits multiple *wujuh* (2 / 4 / 6 counts) for Badal, Aarid, and Lin; the analyser locks to the canonical *qasr* (2 counts) so each recitation is graded against a single specific wajh. The acceptance window widens for the 6-count rules to absorb measurement noise.

5.3 Pipeline

Step 1 — Reference Parsing

`parse_reference(text)` in `madd.py` walks each word character-by-character, detects every madd letter (alif, waw, ya, dagger-alif, small waw/ya, alif-madda), classifies its hukm using vowel and post-letter context, and emits a `MaddSpan` carrying the expected counts and tolerance.

Step 2 — Word Alignment

The reference words are aligned to ASR words via Levenshtein distance over diacritic-stripped letters (`strip_diacritics` also normalises alif-wasl, alif-maqsurah, and tamarbuta to the forms the model emits). Reference words that the ASR didn't recognise emit a `major omission` error.

Step 3 — CTC Forced Alignment

`madd_alignment.align_reference` runs the model's CTC head over the audio (one extra forward pass, no RNNT cost), tokenises the trimmed reference with the model's SentencePiece tokenizer, and forced-aligns via `torchaudio.functional.forced_align`. Output is a list of `CharSpan(global_char_index, char, start, end)` giving real per-letter timings.

Step 4 — Haraka Calibration

`estimate_haraka_ms` computes one *haraka* in milliseconds from the recording itself. The primary signal is total audio duration divided by total emitted characters (a speech-rate proxy); two corroborating medians (all-char durations and Tabii-madd durations divided by 2) are logged but not blended. The result is clamped to [120, 600] ms.

Step 5 — Grading

For each `MaddSpan`, the analyser intersects its character range with the CTC spans (or falls back to linear interpolation across the matched ASR word), measures the duration in milliseconds, divides by `haraka_ms` to get observed counts, and grades against the rule's window: outside the window → `major`; inside but $|\Delta| > 0.7$ from expected → `minor`; otherwise no error.

5.4 Waqf Detection

A word is treated as a *waqf* (pause) when followed by ≥ 300 ms of silence in the ASR alignment, or when it is the last word in the transcript. Ayah boundaries are deliberately

not used — only acoustic silence drives the decision. **aarid** and **lin** spans are conditional on **waqf**; a word-final madd letter emits both an **aarid** candidate (requires **waqf**) and a **tabii** fallback (skip-if-waqf), and exactly one of the two ever grades.

5.5 Output Schema

Each detected mistake is a **TajweedError** JSON object containing the word, its index, the rule (**Al-Madd**), the Arabic sub-rule name, severity (**minor** / **major**), expected and detected counts, a one-sentence explanation, and a millisecond timestamp range. The full payload also surfaces **haraka_ms** so the client can display the calibrated value.

6. Setup and Deployment

6.1 Production: One-Shot Droplet Provisioning

Production runs on a single DigitalOcean droplet provisioned by the helper script `scripts/provision-droplet.sh`. From a fresh Ubuntu 22.04 root shell:

Provisioning Command

```
DROPLET_IP="<DROPLET_IP>"
SITE_HOST="${DROPLET_IP//./-}.nip.io"

ssh -i ~/.ssh/tahkik_do root@$DROPLET_IP \
    "curl -fsSL https://raw.githubusercontent.com/\
mohamedbenhadjer/tahkik-server/main/scripts/provision-droplet.sh \
    | SITE_HOST=$SITE_HOST bash"
```

The script is idempotent and does:

1. Create a 4 GB swapfile and tune `vm.swappiness=10`.
2. Install Docker Engine + Compose plugin (with JSON log rotation).
3. Detect a GPU (via `lspci`) and install the NVIDIA Container Toolkit if present.
4. Clone the repository to `/opt/tahkik-server`.
5. Write `.env` with `SITE_HOST`, `MODEL_ID`, `NEMO_DEVICE`, `HEALTH_TOKEN` (auto-generated if not supplied), and optional `HF_TOKEN`.
6. Build all three containers and bring the stack up. On a GPU droplet the `docker-compose.gpu.yml` overlay is applied automatically.

First boot takes 10–15 minutes: `nemo_toolkit[asr]` pulls roughly 1.5 GB of wheels and the model checkpoint is around 500 MB.

6.2 Continuous Deployment

The repository's `.github/workflows/deploy.yml` watches `main` and triggers a redeploy via SSH using the `appleboy/ssh-action` runner. On the droplet it runs:

Listing 6.1: GitHub Actions redeploy script

```

1 cd /opt/tahkik-server
2 git fetch origin main
3 git reset --hard origin/main
4 export GIT_SHA=$(git rev-parse --short HEAD)
5 export DEPLOYED_AT=$(date -u +%Y-%m-%dT%H:%M:%SZ)
6 docker compose pull caddy || true
7 if lspci | grep -i nvidia >/dev/null 2>&1; then
8     docker compose -f docker-compose.yml -f docker-compose.gpu.yml
9         \
10             up -d --build
11 else
12     docker compose up -d --build
13 fi
14 docker image prune -f

```

GIT_SHA and DEPLOYED_AT are baked into the Go binary via `-ldflags` so the `/api/v1/system-health` response advertises the running build.

6.3 Local Development

For local iteration the Go and Python servers can run without Docker:

Step 1 — Python Environment

```

cd python
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

```

Step 2 — Start Inference Server

```
python inference_server.py -port 8081
```

Wait for the `[inference] READY` line on stdout.

Step 3 — Start Go Server

```

export INFERENCE_SERVER_URL=http://localhost:8081
export PORT=8080
go run ./cmd/api

```

The local Python entry point (`python/inference_server.py`) is the legacy Whisper development server using `benhadjermed/tahkik-small-warsh`. The production stack always uses the NeMo container at `python/hf_space/`.

6.4 Environment Variables

Table 6.1: Configuration environment variables

Variable	Default	Purpose
SITE_HOST	—	Public hostname Caddy issues a TLS cert for
DASHBOARD_ORIGIN	http://localhost:5173	CORS origin allowed by Caddy
MODEL_ID	nvidia/stt_ar_fc...	NeMo model identifier
NEMO_DEVICE	cpu	cpu or cuda
TORCH_NUM_THREADS	4	CPU thread cap (irrelevant on GPU)
INFERENCE_SERVER_URL	http://localhost:8081	Backend URL seen by the Go server
HF_TOKEN	—	Bearer token for HF access (private repos)
HEALTH_TOKEN	—	Bearer token for /api/v1/system-health
PORT	8080	Go server listen port
COMPOSE_PROJECT_NAME	tahkik-server	Compose label used by container enumeration

6.5 Hardware Recommendations

Table 6.2: Tested hardware tiers

Tier	Hardware	Cold Start	Partial / Final p50
Development	CPU laptop	~30 s	~5 / ~10 s
Production CPU	DO s-4vcpu-8gb	~60 s	3–8 / 8–15 s
Production GPU	NVIDIA T4 / L4	~40 s	<1 / 1–2 s

7. API Reference

7.1 Go Server Endpoints (Client-Facing)

7.1.1 POST /api/v1/evaluate

Upload an audio file for Arabic Quranic recitation transcription, optionally with a reference text to enable madd analysis.

Request Format

Content-Type: multipart/form-data

Required field: audio (file — .wav, .mp3, .m4a, .flac, .ogg; max 50 MB)

Optional field: reference_text (string — bare text *or* JSON-encoded array [{"surah":1,"ayah":1,"text":"..."}, ...])

Success Response (200 OK) — without reference_text

```
{
  "status": "success",
  "data": {
    "transcription": "<Arabic transcription text>",
    "confidence_score": 0.9423,
    "processing_time_ms": 1350
  }
}
```

Success Response (200 OK) — with reference_text

```
{
  "status": "success",
  "data": {
    "transcription": "...",
    "confidence_score": 0.93,
    "processing_time_ms": 2100,
    "haraka_ms": 280,
    "errors": [
```

```

    {
      "word": "...",
      "word_index": 3,
      "rule": "Al-Madd",
      "sub_rule": "...",
      "severity": "minor",
      "expected": "2 counts",
      "detected": "~1.4 counts",
      "explanation": "Natural elongation of 2 counts...",
      "timestamp": {"start": 1.872, "end": 2.144}
    }
  ]
}

```

Error Response (4xx / 5xx)

```
{ "status": "error", "message": "unsupported audio format: .xyz" }
```

7.1.2 GET /api/v1/system-health (Bearer-token gated)

Returns the aggregated host + container + inference snapshot described in Section 3.2.5.

Response (200 OK) — shape only

```

{
  "status": "ok", // ok | loading | degraded
  "host": {
    "hostname": "tahkik-nemo",
    "uptime_seconds": 81234,
    "cpu_percent": 14.2,
    "mem_used_mb": 3210, "mem_total_mb": 7890,
    "swap_used_mb": 0, "swap_total_mb": 4096,
    "disk_used_gb": 27, "disk_total_gb": 160,
    "load_1m": 0.71, "load_5m": 0.65
  },
  "containers": [
    { "name": "caddy", "status": "running", "up_since": "..."},
    { "name": "go-server", "status": "healthy", "up_since": "..."},
    { "name": "python", "status": "healthy", "up_since": "..."}
  ]
}

```



```

],
"inference": { "requests_total": ..., "partial_p95_ms": ..., ... },
"model":      { "id": "...", "device": "cpu", "loaded": true, ... },
"version":    { "git_sha": "abc1234", "deployed_at": "2026-..." }
}

```

7.1.3 GET /api/v1/system-health/ws (WebSocket)

WebSocket upgrade of the above endpoint. Authentication is via `?token=<HEALTH_TOKEN>` on the URL because browsers cannot set custom headers on a WebSocket handshake. The server pushes a fresh snapshot every 2 seconds and responds to client close frames cleanly.

7.1.4 GET /health

Health Check Response (200 OK)

```
{"status": "ok"}
```

7.2 Response Fields

Table 7.1: data object fields on `/api/v1/evaluate`

Field	Type	Description
<code>transcription</code>	string	Arabic transcription of the recitation
<code>confidence_score</code>	float	Mapped RNNT log-prob score in $[0, 1]$
<code>processing_time_ms</code>	int	Time in model (ms), excluding network
<code>haraka_ms</code>	int	Auto-calibrated one-haraka length (only when <code>reference_text</code> was sent)
<code>errors</code>	list	Detected <code>TajweedError</code> objects (only when <code>reference_text</code> was sent)

7.3 Flutter Integration Example

Listing 7.1: Dart/Flutter API call example

```

1 final dio = Dio(BaseOptions(baseUrl: 'https://<site-host>'));
2

```

```
3 Future<Map<String, dynamic>> evaluate(File audioFile,
4                                     {String? referenceText})
5                                     async {
6     final formData = FormData.fromMap({
7         'audio': await MultipartFile.fromFile(
8             audioFile.path,
9             filename: path.basename(audioFile.path),
10        ),
11        if (referenceText != null) 'reference_text': referenceText,
12    });
13    final response = await dio.post('/api/v1/evaluate', data:
14        formData);
15    return response.data as Map<String, dynamic>;
16 }
```

8. Troubleshooting

8.0.1 Inference Server Unreachable / 502 from Caddy

Diagnosis

Caddy is up but the Go server cannot reach the Python container. Either the Python container is still loading the NeMo model (cold start can take 10 min on first boot — the wheel install plus model download) or the container has crashed.

Resolution

```
docker compose -f /opt/tahkik-server/docker-compose.yml logs python
-tail=50. Watch for [inference] model ready (warmup <ms>). If you see Python-
level tracebacks, restart with docker compose restart python.
```

8.0.2 Model Download Timeout

Diagnosis

First-time model download from HuggingFace can take several minutes on a slow link. The Python server prints `[inference] loading NeMo model ...` and stays silent until the checkpoint is fully fetched into `/cache/nemo`.

Resolution

Wait for the download to complete. Subsequent restarts use the cached weights in the `nemo_cache` named volume.

8.0.3 system-health Returns 503 / 401

Diagnosis

503 means `HEALTH_TOKEN` is unset in the environment (the endpoint fails closed). 401 means the `Authorization: Bearer <token>` header doesn't match the value baked into `.env`.

Resolution

Verify `grep HEALTH_TOKEN /opt/tahkik-server/.env` on the droplet and ensure the same value is wired into the dashboard's `VITE_TAHKIK_HEALTH_TOKEN`.

8.0.4 Partial Latency Above the Window**Diagnosis**

Each partial transcribes 8 s of audio. If inference itself takes longer than 8 s, the stream falls behind real-time and the user experience degrades.

Resolution

In order of preference: (1) reduce `PARTIAL_WINDOW_S` in `python/hf_space/main.py` to 5; (2) raise `TORCH_NUM_THREADS` to the droplet's vCPU count; (3) move to a GPU droplet and apply `docker-compose.gpu.yml`.

9. Performance Metrics

9.1 Inference Latency by Hardware

The inference latency varies significantly between CPU and GPU droplets and between the partial-window and final inference paths. Measurements below were taken on the production droplet (`s-4vcpu-8gb`) for the CPU rows and on a single NVIDIA T4 for the GPU rows, using a 10-second Quranic audio sample (16 kHz mono 16-bit WAV).

Table 9.1: Average inference latency by hardware (p50 / p95)

Path	Hardware	p50	p95	Throughput
Partial (8 s window)	CPU 4-vcpu	~3.5 s	~7.0 s	~0.28 req/s
Final (10 s audio)	CPU 4-vcpu	~8.5 s	~15 s	~0.12 req/s
Partial (8 s window)	T4 GPU (FP16)	~0.6 s	~0.9 s	~1.7 req/s
Final (10 s audio)	T4 GPU (FP16)	~1.2 s	~1.8 s	~0.85 req/s

Key Observations

- The RNNT decoder dominates wall-time on CPU; GPU acceleration yields a ~6–8× speedup on the same checkpoint.
- Final inference is ~2–3× slower than the partial because it processes the full (capped at 60 s) buffer rather than the trailing 8 s.
- The Go API gateway introduces negligible overhead (<5 ms per request) — multipart parsing and HTTP proxying only.
- Madd analysis adds 200–600 ms on top of the inference time when `reference_text` is provided (one extra CTC forward pass for forced alignment).

9.2 Processing Time Breakdown

For a typical 10-second audio file with `reference_text` provided (production CPU droplet, end-to-end through Caddy + Go + Python):

Table 9.2: Request processing time breakdown (10s audio + madd, CPU)

Stage	Duration (ms)	Percentage
Caddy TLS termination & routing	~10	0.1 %
Go server parsing & validation	~5	0.1 %
Network transfer (Go → Python)	~15	0.2 %
Audio resampling (<code>librosa</code>)	~45	0.5 %
RNNT inference (NeMo, FastConformer)	~8 200	90.0 %
CTC forced alignment	~500	5.5 %
Madd analyser	~120	1.3 %
JSON serialisation & return path	~30	0.3 %
Total end-to-end	~9 100	100 %

Model inference constitutes 90 % of total processing time, confirming that further performance work should target the ML backend (GPU upgrade, smaller checkpoint) rather than the API or analysis layers.

10. Scalability and Load Handling

10.1 Current Concurrency Model

The Go API server handles concurrent HTTP connections natively through goroutines. The inference backend, however, intentionally serialises all model invocations via a module-level `asyncio.Semaphore(1)` plus a per-connection `asyncio.Lock` on the WebSocket path:

Concurrency Limitations

- **Global inference semaphore:** One inference at a time across the entire Python process. Prevents CPU over-subscription and non-deterministic output ordering. Concurrent requests are queued.
- **Single-worker Uvicorn:** The container runs one Uvicorn worker. Multi-process scaling would require multiple model copies — ~ 500 MB of CPU RAM each — which is infeasible on the baseline 8 GB droplet.
- **Model memory footprint:** The FastConformer checkpoint occupies ~ 500 MB of CPU RAM (or ~ 1.2 GB of GPU VRAM in FP16). Loading multiple instances scales linearly.

10.2 Observed Behaviour Under Load

Under concurrent load on the CPU production droplet the system exhibits a predictable serial-queue pattern:

Table 10.1: Server behaviour under concurrent requests (CPU, 10 s audio without madd)

Concurrent Requests	Avg. Response Time	Re- Behaviour
1	~8.5 s	Normal
3	~25 s	Queued (serial execution)
5	~42 s	Queued, increased wait
10+	~85 s+	High risk of client-side timeout (5 min ceiling at the Go layer)

10.3 Scalability Improvement Strategies

Recommended Scalability Enhancements

- **GPU upgrade:** A single T4/L4 GPU droplet cuts final latency below 2 s and lifts effective single-stream throughput by $\sim 7\times$. The fastest unit-of-work improvement.
- **Horizontal scaling:** Run multiple droplets behind a load balancer. Each droplet is fully self-contained (Caddy issues its own cert), so adding capacity is one-shot.
- **Multi-worker Uvicorn (with care):** Launch Uvicorn with `-workers N` only when RAM permits one model copy per worker. The semaphore stays per-process so concurrency is bounded by worker count.
- **Bounded request queue:** Add a token-bucket or in-memory queue at the Go layer to reject excess work with `503 Service Unavailable` rather than accumulating unbounded latency.
- **Batching on GPU:** On GPU hardware, batch multiple audio inputs in one forward pass to amortise the fixed kernel-launch overhead.

11. Security Considerations

11.1 Transport Layer

Caddy terminates TLS using a Let's Encrypt certificate issued automatically on the first HTTPS request. Plain-text HTTP traffic on port 80 is redirected by Caddy's default behaviour. The Go and Python containers *never* bind to a public address — they expose ports only within the compose network.

11.2 Input Validation

The Go API gateway enforces multiple layers of input validation before any data reaches the inference backend:

Validation Rules

- **HTTP method:** `/api/v1/evaluate` accepts only POST; other methods return 405 Method Not Allowed.
- **File size limit:** `r.ParseMultipartForm(50 « 20)` enforces a strict 50 MB ceiling. Larger uploads return 400 Bad Request.
- **File-format whitelist:** Only five audio extensions are accepted (`.wav`, `.mp3`, `.m4a`, `.flac`, `.ogg`). Unrecognised extensions are rejected before any read past the multipart header.
- **Limit-read of audio:** The handler wraps the form file in `io.LimitReader(file, 50«20)` so a malformed multipart cannot stream past the declared ceiling.
- **Format check at the backend:** The Python `/evaluate` handler re-checks the extension on the server side as defence-in-depth.

11.3 Token Management

Two distinct secrets are managed exclusively on the server side:

Token Security Architecture

- **HF_TOKEN**: Stored in `/opt/tahkik-server/.env` (mode 600) on the droplet. Injected into the Python container as an environment variable. Used to authenticate to HuggingFace when downloading private model weights. The Flutter app never sees it.
- **HEALTH_TOKEN**: Generated on first provisioning (`head -c 32 /dev/urandom | base64, 40 chars`). Required as a Bearer token on every `/api/v1/system-health` call and as a `?token=` query parameter on the WebSocket variant. The Go middleware fails closed (503) if the env var is unset, so the endpoint cannot accidentally become public.

11.4 Docker Socket Exposure

The Go container mounts `/var/run/docker.sock` read-only (`:ro`) to enumerate sibling containers for the system-health endpoint. Read-only mode prevents container creation, image manipulation, or any state-changing API call. The distroless `nonroot` user (UID 65532) is added to the host's docker group (GID 999) at runtime via `group_add` so the socket is readable without granting root inside the container.

11.5 Abuse Prevention

Table 11.1: Current protections and proposed enhancements

Measure	Status	Description
TLS at edge (Let's Encrypt)	Implemented	All client traffic encrypted by default
File size cap (50 MB)	Implemented	Prevents oversized-upload resource exhaustion
Format whitelist	Implemented	Blocks arbitrary file uploads
Inference semaphore	Implemented	Natural throttle: 1 concurrent inference
HTTP timeout (5 min)	Implemented	Prevents indefinite connection holding
Bearer-token gate on system-health	Implemented	Operator-only endpoint
Distroless + nonroot Go container	Implemented	No shell, no package manager, minimal attack surface
Rate limiting	Proposed	Token-bucket at Caddy or Go layer
Per-client API keys	Proposed	Identify and throttle individual mobile users

12. Request Lifecycle

12.1 End-to-End Request Flow

The following describes the complete lifecycle of a single audio-evaluation request, from client submission to response delivery.

Step 1 — Client Submission

The Flutter mobile application captures or selects an audio recording, optionally pairs it with the ayah text the user was asked to recite, and sends it as a `multipart/form-data` POST request to `https://<SITE_HOST>/api/v1/evaluate`. The audio is attached under the `audio` form field; the reference (if any) under `reference_text`.

Step 2 — Caddy TLS & Routing

Caddy terminates the TLS handshake, runs the CORS preflight if applicable, and matches the request against the `@api` path matcher. The request is reverse-proxied to `go-server:8080` on the internal compose network. The CORS `Access-Control-Allow-Origin` response header is set from `$DASHBOARD_ORIGIN`.

Step 3 — Go Server Validation

The Go server parses the multipart form (enforcing the 50 MB limit), extracts the `audio` field, validates the file extension against the whitelist, reads the binary data into memory through a limit-reader, and captures the optional `reference_text`.

Step 4 — Inference Forwarding

The Go server constructs a fresh `multipart/form-data` request containing both `audio` and `reference_text` (when supplied), targets `$INFERENCE_SERVER_URL/evaluate`, and attaches the `Authorization: Bearer $HF_TOKEN` header if available. The HTTP client uses a 5-minute timeout.

Step 5 — Backend Audio Processing

The Python FastAPI handler receives the upload, writes it to a temp file under `/tmp`, and loads it via `librosa.load(sr=16000, mono=True)`. The audio array is clamped to the last 60 seconds.

Step 6 — Model Inference

The array is passed to `model.transcribe(...)` under the global inference semaphore. When `reference_text` is present, the call requests timestamps and hypotheses so per-word and per-character timings can be extracted.

Step 7 — Optional Madd Analysis

The CTC head is run for forced alignment, the analyser computes per-rule errors, and the response payload is augmented with `errors[]` and `haraka_ms`.

Step 8 — Response Assembly

The temp file is deleted in a `finally` block. The JSON result is returned to the Go server, which wraps it in the standard `{"status": "success", "data": {...}}` envelope and forwards it back through Caddy to the Flutter client.

12.2 Concurrency Patterns

Different components employ distinct concurrency strategies, each chosen for the workload it sees:

Table 12.1: Concurrency mechanisms across components

Component	Pattern	Rationale
Caddy	Event loop (Go native)	High-throughput TLS reverse proxy
Go API server	Goroutines	Lightweight; one per request
Python REST	Single Uvicorn worker	Backed by one model copy
Python WS	<code>asyncio</code> tasks	Bounded by per-connection lock
NeMo inference	<code>Semaphore(1)</code>	Single CPU/GPU shared resource

13. Failure Handling

13.1 Error Categories and Responses

The server distinguishes several categories of failure, each returning a specific HTTP status code and error message:

Table 13.1: Failure modes and corresponding HTTP responses

Failure Mode	HTTP Status	Response Action
Missing audio field	400	"audio field required"
Wrong HTTP method	405	"method not allowed"
Oversized upload	400	"file too large"
Unsupported audio format	400	Reject with file extension info
Inference timeout (5 min)	504	Return timeout error
Backend (Python) returns 5xx	502	Wrap and forward error
Caddy cannot reach Python (cold)	502	Generic gateway error
HEALTH_TOKEN unset	503	system-health endpoint fails closed
Wrong Bearer token	401	system-health rejects
Internal Python exception	500	Log + return generic message

13.2 Error Response Format

All client-facing errors at the Go layer use a uniform envelope:

Listing 13.1: Uniform error response

```
1 {  
2   "status": "error",  
3   "message": "<human-readable error description>"  
4 }
```

This matches the success envelope's shape (`{"status": "success", "data": {...}}`) so clients can use a single discriminator field to branch.

13.3 Resilience Mechanisms

Implemented Resilience Patterns

- **Per-task exception isolation:** Errors in a WebSocket partial-window task are caught, logged with full traceback, and discarded — they cannot tear down the WebSocket connection.
- **Temp-file cleanup:** The `/evaluate` handler deletes uploaded audio in a `finally` block regardless of whether inference succeeded or threw.
- **Per-container healthchecks:** Each container has a `HEALTHCHECK` directive (Go uses the `healthcheck` sub-command of its own binary; Python uses an inline `urllib` probe; Caddy's image healthcheck is sufficient).
- **restart: unless-stopped:** All three services restart automatically on crash unless explicitly stopped by an operator.
- **Cached system-health snapshot:** The 5-second cache absorbs traffic spikes from the dashboard's poll loop without thrashing `/proc` or Docker.
- **System-health WS keepalive:** The WebSocket sends `ping` frames and the server replies with `pong`; a missed `pong` tears down the connection and the dashboard reconnects.
- **Reset/stop session control:** The WebSocket protocol exposes explicit `reset` and `stop` messages; both cancel in-flight tasks and `reset` bumps a generation counter so stale partials are discarded silently.

14. Logging and Monitoring

14.1 Logging Strategy

The server employs a layered logging approach with structured log entries at each tier:

Table 14.1: Logging implementation by component

Component	Log Output	Captured
Caddy	Container stdout (Docker JSON)	Access log + cert events
Go server	log package → stdout	Request lifecycle, errors, version banner
Python REST	print(..., flush=True)	Model loading, request timing
Python WS	Same	Connect/disconnect, partial/final emit, errors
Madd analyser	print(..., flush=True)	Word alignment, haraka calibration, span counts

All container logs are captured by Docker’s `json-file` driver with 20 MB rotation across 5 files (configured in `/etc/docker/daemon.json` by the provisioning script). Operators can tail any container with `docker compose logs -f <service>`.

14.2 Metrics Exposure

The Python container maintains lightweight in-process counters and rolling latency dequeues (200 samples each for partial and final paths). These are returned by `GET /health`:

Listing 14.1: Metrics snapshot returned by Python `/health`

```
1 {
2   "inference": {
3     "requests_total":      123,
```



```

4      "requests_failed":      2,
5      "ws_connections_total": 45,
6      "ws_connections_active": 3,
7      "partial_p50_ms":      3450,
8      "partial_p95_ms":      7100,
9      "final_p50_ms":        8200,
10     "final_p95_ms":         14500,
11     "last_request_at":      "2026-...",
12     "last_error_at":        null,
13     "last_error":           null
14 }
15 }
```

The Go server's `/api/v1/system-health` aggregator augments this with host metrics, container status, and version stamps for the operations dashboard.

14.3 Observability Roadmap

Proposed Observability Enhancements

- **Prometheus exposition:** Add a `/metrics` endpoint exporting the same in-process counters in Prometheus format for scraping by an external collector.
- **Structured JSON logs:** Replace the current line-based logs with single-line JSON entries so log shippers (Vector, Fluent Bit) can index them.
- **Request tracing:** Generate a unique trace ID at the Caddy layer and propagate through Go and Python logs to correlate per-request events.
- **DigitalOcean alert policies:** External backstop for the in-dashboard alerts — four threshold policies (CPU, memory, disk, droplet-down) documented in `docs/do-alerts-setup.md`.
- **Centralized error reporting:** Forward Python tracebacks and Go panics to a Sentry or similar aggregator.

14.4 Dashboard Integration

The Tahkik admin dashboard polls `/api/v1/system-health` every 5s for the snapshot view and opens a WebSocket to `/api/v1/system-health/ws` for the live metrics page. Both endpoints require the `HEALTH_TOKEN`; the dashboard reads it from its own `VITE_TAHKIK_HEALTH_TOKEN` environment variable and stores it only client-side (no propagation to third parties).

15. Conclusion

15.1 Summary

This report has documented the architecture, implementation, and operational concerns of the Tahkik inference server in its current NeMo-based form. The system delivers production-grade Arabic Quranic recitation transcription and Tajweed-rule analysis to the Flutter mobile application through a three-container Docker Compose stack: Caddy (TLS + routing), a Go API gateway (validation + system-health aggregation), and a Python FastAPI server hosting NVIDIA’s FastConformer Hybrid CTC/RNNT model.

15.2 Key Achievements

Project Highlights

- **Engine upgrade:** Successfully migrated from the legacy Whisper / CTranslate2 stack to NVIDIA NeMo FastConformer with no Flutter-side code change (only an HF_SPACE_URL swap).
- **Quranic-domain accuracy:** 6.55 % WER on the EveryAyah benchmark on the same canonical checkpoint that drives the inference path.
- **Real-time streaming:** `/ws/stream` delivers partial transcriptions on an 8-second sliding window with <1 s p50 latency on a single T4 GPU.
- **Madd-rule grading:** The analyser implements eight Warsh/Azraq madd rules with auto-calibrated haraka length and CTC-derived per-letter timings.
- **One-command deploy:** `provision-droplet.sh` stands up a fresh production droplet end-to-end (Docker + TLS + model cache) in 10–15 minutes.
- **Operator dashboard:** `/api/v1/system-health` surfaces host, container, and inference metrics under a Bearer-token-gated endpoint with a WebSocket variant for live updates.

15.3 Future Enhancements

Roadmap

- **Performance**
 - Migrate to GPU droplets (T4/L4) as traffic warrants.
 - Quantise the NeMo checkpoint to INT8 for CPU inference (requires NVIDIA’s quantisation tooling).
 - Batch concurrent inferences on GPU.
- **Reliability**
 - Add rate limiting at the Caddy layer.
 - Add a bounded request queue at the Go layer with explicit 503 responses.
 - Wire Sentry or equivalent for error aggregation.
- **Tajweed analysis**
 - Extend rules beyond madd: idgham, ikhfa, qalqala, ghunna detection.
 - Add tafkhim/tarqiq detection for the lam and ra letters.
 - Multi-wajh grading mode (accept 2 / 4 / 6 counts) for Badal, Aarid, Lin instead of locked qasr.
- **Operations**
 - Multi-region deployments behind a global load balancer.
 - Automated blue/green deploys via the GitHub Actions workflow.
 - Prometheus + Grafana stack for long-term metrics retention.

15.4 Final Remarks

Production-Ready Inference Stack

The Tahkik server stack runs reliably on a single-host Docker Compose deployment, with a production CPU baseline at s-4vcpu-8gb and a GPU-overlay one-liner away.

The architecture cleanly separates TLS, REST routing, and ML inference, making each layer independently upgradable.

References

Bibliography

- [1] NVIDIA Corporation. *NeMo: A Toolkit for Conversational AI*.
<https://github.com/NVIDIA/NeMo>
- [2] Rekesh, D., Koluguri, N. R., *et al.* (2023). *Fast Conformer with Linearly Scalable Attention for Efficient Speech Recognition*. ASRU 2023.
<https://arxiv.org/abs/2305.05084>
- [3] NVIDIA. *stt_ar_fastconformer_hybrid_large_pcd_v1.0 model card*.
https://huggingface.co/nvidia/stt_ar_fastconformer_hybrid_large_pcd_v1.0
- [4] Graves, A. (2012). *Sequence Transduction with Recurrent Neural Networks*. ICML 2012 Representation Learning Workshop.
<https://arxiv.org/abs/1211.3711>
- [5] Graves, A., Fernández, S., Gomez, F., Schmidhuber, J. (2006). *Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks*. ICML 2006.
- [6] PyTorch Team. *Forced Alignment with Wav2Vec2 / CTC* — *torchaudio.functional.forced_align*.
https://pytorch.org/audio/stable/generated/torchaudio.functional.forced_align.html
- [7] Ramírez, S. *FastAPI: Modern, fast (high-performance) web framework*.
<https://fastapi.tiangolo.com>
- [8] Christie, T. *Uvicorn — The lightning-fast ASGI server*.
<https://www.uvicorn.org>
- [9] Matt Holt et al. *Caddy — The Ultimate Server with Automatic HTTPS*.
<https://caddyserver.com>
- [10] The Go Authors. *Package net/http*.
<https://pkg.go.dev/net/http>

- [11] Coder. *github.com/coder/websocket — minimal and idiomatic WebSocket library for Go*.
<https://github.com/coder/websocket>
- [12] Docker, Inc. *Docker Engine documentation*.
<https://docs.docker.com/engine/>
- [13] Docker, Inc. *Docker Compose specification*.
<https://docs.docker.com/compose/>
- [14] McFee, B. *librosa: Audio and music signal analysis in Python*.
<https://librosa.org>
- [15] *Tariq al-Azraq (Warsh ‘an Nafi’ transmission)* — riwayat documentation in the Tajweed literature. Standard reference works on Madinah Mushaf and Tariq Azraq variants.
- [16] Ben Hadj Mohamed. *Tahkik Server Source Code Repository*.
<https://github.com/mohamedbenhadjer/tahkik-server>
- [17] EveryAyah. *Quranic Audio Dataset*.
<https://everyayah.com>

End of Report